



Manual Técnico de Desarrollo

5 de septiembre de 2026

Panorama

Qué es esto

gestISP es un sistema de gestión para proveedores de internet: clientes y contratos, facturación mensual, recaudo, almacén, órdenes técnicas, equipos de red (OLT/ONT/PPPoE) y documentación de la planta de fibra. Incluye facturación electrónica ante la DIAN de Colombia.

Es una aplicación Laravel monolítica, renderizada en servidor con Blade, sin API pública ni cliente de JavaScript propio más allá de lo que traen AdminLTE y DataTables.

Lo primero que hay que saber

Tres cosas que gobiernan todo el código y conviene tener claras antes de abrir un archivo:

1. **Todo está en español.** Nombres de métodos, variables, comentarios, mensajes, pruebas. Las clases usan nombres en inglés porque son de framework (`InvoiceGenerator`, `XadesSigner`), pero sus métodos y su documentación van en español. No mezcle.
1. **Los comentarios explican el PORQUÉ, no el qué.** El código ya dice qué hace. Los comentarios de este proyecto documentan la decisión: por qué se eligió así, qué pasa si se cambia, qué error se cometió antes. Muchos son largos a propósito. Léalos antes de "simplificar" algo.
1. **Todo queda en trazabilidad.** Cualquier cosa que un usuario cree, modifique o borre tiene que quedar registrada. No es opcional.

Pila tecnológica

	Versión
PHP	8.1+ (se desarrolla en 8.3)
Laravel	10.x
Base de datos	MySQL / MariaDB
Interfaz	Blade + AdminLTE 3 + DataTables
Permisos	spatie/laravel-permission 6
PDF	barryvdh/laravel-dompdf
Excel	maatwebsite/excel
Mikrotik	evilfreelancer/routeros-api-php
SSH	phpseclib3
Códigos de barras y QR	milon/barcode
Pruebas	PHPUnit 10

No hay extensión `soap`. Los sobres SOAP de la DIAN se arman a mano sobre Guzzle. No la dé por disponible.

SNMP se usa mediante las funciones nativas de PHP; el servidor las necesita.

Montar el entorno

Requisitos

- PHP 8.1 o superior con las extensiones habituales de Laravel, más `snmp` y `openssl`.
- MySQL o MariaDB.
- Composer y Node (Node solo para utilidades, no hay build de front obligatorio).

En Windows se desarrolla con **Laragon**. En producción, Linux.

Puesta en marcha

```
composer install
cp .env.example .env
php artisan key:generate
```

Configure la base en `.env` y después:

```
php artisan migrate
php artisan db:seed --class=RoleSeeder
php artisan permissions:sync
php artisan dian:catalogos
```

- `RoleSeeder` crea los cuatro roles y **todos** los permisos.
- `permissions:sync` crea los permisos que falten en una base existente. **Hay que correrlo después de cada despliegue que añada permisos**, o el menú no mostrará las opciones nuevas.
- `dian:catalogos` carga los catálogos fiscales de la DIAN (municipios, tipos de documento, responsabilidades).

Variables de entorno que importan

Variable	Para qué
<code>QUEUE_CONNECTION</code>	<code>database</code> en producción. Con <code>sync</code> los trabajos corren en la petición.
<code>DIAN_ENDPOINT</code>	Fuerza la URL del servicio DIAN para toda la instalación. Normalmente vacío.
<code>MAIL_*</code>	Envío de correos.
<code>WHATSAPP_*</code>	Integración de WhatsApp. Sin configurar, el canal simula el envío.

OpenSSL en Windows

`openssl_pkey_new()` falla en Windows si no encuentra `openssl.cnf`. El código lo busca en varios sitios (`OPENSSL_CONF`, `<php>/extras/ssl/openssl.cnf`, `/etc/ssl/openssl.cnf`). Si aun así falla, define `OPENSSL_CONF`. En Linux no ocurre.

Arquitectura

Dónde va cada cosa

Carpeta	Qué contiene
app/Http/Controllers	Reciben la petición, validan, delegan. No llevan lógica de negocio.
app/Billing/Services	Facturación: emisión, numeración, pagos, notas, retenciones.
app/Billing/Dian	Todo lo de la DIAN: XML, CUFE, firma, transporte.
app/Services	Red, importaciones, auditoría, copias, numeración de documentos.
app/Reports	Informes y el de completitud fiscal.
app/Models	Eloquent. Relaciones, alcances y reglas propias del dato.
app/Tenancy	Multiempresa: contexto y filtrado.
app/Jobs	Trabajos en cola.
app/Listeners	Reaccionan a eventos de dominio.
app/Console/Commands	Tareas de consola y programadas.
resources/views/gestisp	Vistas, una carpeta por módulo.

La regla del controlador delgado

Un controlador valida, llama a un servicio y devuelve una vista o una redirección. Si empieza a tener condicionales de negocio, esa lógica va a un servicio.

Esto no es dogma: es que la lógica en el controlador solo se puede probar por HTTP, y las reglas de facturación necesitan probarse directamente.

Multiempresa

El modelo

```
Company (el contribuyente: un NIT)
└─ Branch (una sede)
    └─ clientes, contratos, facturas, almacén, técnicos...
```

Antes de esto, "sucursal" y "contribuyente" eran lo mismo. Quedan restos de esa época; si encuentra código que trata a la sucursal como emisor fiscal, probablemente sea uno.

CurrentContext

Es un singleton que guarda desde dónde está trabajando el usuario: empresa, sucursales accesibles y sucursal activa.

```
$contexto = app(CurrentContext::class);
$contexto->companyId();      // la empresa
$contexto->branchId();       // la sucursal activa (null en consolidado)
$contexto->branchIds();      // todas a las que tiene acceso
$contexto->esConsolidado();  // ¿está viendo varias a la vez?
```

Métodos que se usan mucho:

- `branchParaEscritura($solicitada)` — decide en qué sucursal se guarda algo, validando que el usuario pueda.
- `limitarSucursales($query)` — aplica el filtro de sucursal a una consulta.
- `sinContexto($callback)` — ejecuta algo sin el filtro. Para tareas de consola.

El contexto lo repone en cada petición el middleware `SetCompanyContext`, desde la sesión.

BelongsToCompany

Un trait con un **global scope** que filtra por `company_id`.

Filtra por empresa, NO por sucursal. El filtrado por sucursal es manual, consulta por consulta. Es el error más fácil de cometer: dar por hecho que el trait ya limita a la sede.

Para saltarse el filtro: `Modelo::sinFiltroDeEmpresa()` o `withoutGlobalScopes()`.

Un desfase que conviene conocer

La tabla pivote `user_branch` guarda el rol del usuario en cada sucursal. La migración a empresas no la tocó, así que sigue siendo el punto de unión entre usuario y sede. Funciona, pero no está modelada como el resto.

Convenciones del proyecto

Rutas

Todas cuelgan del prefijo `gestisp/`. **Nunca escriba una URL a mano**, ni en PHP ni en JavaScript: use `route('nombre')`. En las vistas, páselas a JavaScript desde Blade.

Vistas y listados

Los listados usan **DataTables** con procesamiento en servidor. El patrón es: una ruta para la vista y otra para los datos.

Los formularios que van dentro de un modal y usan Select2 necesitan `dropdownParent`, o el desplegable queda debajo del modal.

Trampas de Blade

Dos que ya han costado tiempo:

- `@php` en línea con comillas escapadas **compila y revienta al renderizar**. Si necesita lógica, sáquela a la vista con una variable.
- Un `@include` de un parcial que espera una variable falla si no se la pasa. Use `['x' => $x ?? null]`.

Mensajes y traducciones

Están en `resources/lang/es`. Las páginas de error y las alertas son propias.

Modelo de datos

Las tablas por dominio. Hay 108 migraciones y unas 75 tablas.

Estructura

companies, branches, user_branch, users, roles, permissions

Comercial

clients, client_tax_responsibilities, contracts, contract_comments, plans, services, plan_service, affinity_groups

Facturación

invoices, invoice_items, additional_charges, billing_runs, credit_debit_notes, invoice_numbering_sequences, note_numbering_sequences, document_sequences

Recaudo

payments, payment_batches, payment_retentions, payment_audits, account_credits, cash_registers, cash_register_transactions

DIAN

dian_configurations, dian_certificates, dian_resolutions, numbering_ranges, electronic_documents, document_transmissions, fiscal_catalogs

Almacén y técnicos

warehouses, materials, categories, material_movements, inventories, technical_orders, technical_order_materials, technical_order_verifications

Red

olts, olt_boards, olt_uplinks, pon_ports, olt_port_metrics, onts, ont_metrics, ont_import_runs, pppoe_accounts, pppoe_session_metrics, routers, vlan_olts, srv_profiles, line_profiles

Planta de fibra

optical_networks, network_zones, nap_boxes, nap_ports, fiber_cables, cable_strands, splice_closures, splices, splitters, splitter_outputs

Sistema

audits, user_sessions, failed_logins, notifications, jobs, failed_jobs, pdf_reports

Facturación

La emisión

`App\Billing\Services\InvoiceGenerator` arma una factura a partir de un contrato: toma los servicios de su plan, los cargos adicionales pendientes y el saldo anterior.

`MonthlyBillingRun` recorre los contratos activos y registra la corrida.

Al emitir se dispara el evento `InvoiceIssued`, con dos oyentes en este orden:

1. `GenerateElectronicDocument` — arma el documento DIAN.
2. `NotifyClientInvoiceIssued` — avisa al cliente.

El orden importa: si la factura va a la DIAN, su XML y su CUFE tienen que existir antes de avisarle a nadie.

Lo que se congela al emitir

Un patrón que se repite: la factura guarda copia de lo que era verdad en el momento de emitirla, no una referencia a lo que sea verdad hoy.

- `document_kind` — electrónica o interna.
- `affinity_group_id` — el grupo del contrato en ese momento.
- `concept_label`, `product_code`, `tax_classification` en cada renglón.
- `environment_code` en el documento electrónico.

Una factura no cambia de naturaleza después de emitida.

Numeración: dos series que no se tocan

`InvoiceNumerator::assign()` decide de dónde sale el número según el tipo ya congelado:

- **Electrónica** → `numbering_ranges`, el rango autorizado por la resolución. Si no hay ninguno vigente, **lanza excepción**. No se inventa ni cae a la interna.
- **Interna** → `invoice_numbering_sequences`, serie propia de la sucursal, que sí se crea sola.

Son tablas distintas a propósito: hace imposible que un documento interno gaste un consecutivo autorizado.

El incremento y el control de agotamiento los hace `DocumentNumberService::reservarEn()`, con `lockForUpdate()`. Es el mismo que numera contratos y notas: esa regla no puede tener dos versiones.

Estados de contrato: dos vocabularios

Cuidado con esto. La columna `contracts.status` recibe valores de dos sitios distintos: los que escribe la facturación y los que escribe el formulario manual. **No son el mismo vocabulario.** Use `ContractStatusMap` para interpretarlos; no compare cadenas a mano.

Pagos

- `PaymentRegistrar` — un pago sobre una factura.
- `BatchPaymentRegistrar` — cobro múltiple, **transaccional**: todo o nada.
- `RetentionApplier` — retenciones. Saldan la factura pero **no entran a la caja**.
- `CreditBalanceService` — saldo a favor.

Notas

`NoteIssuer` emite la nota y ajusta el saldo. Numera con **su propia serie**, no con un rango autorizado.

La decisión D6 del plan original decía lo contrario. Es **falsa**: el XML de una nota no lleva `sts:InvoiceControl` y el anexo §12.1 acepta "la numeración establecida por el facturador". Está verificado contra los ejemplos oficiales. No lo "corrija".

Facturación electrónica DIAN

La cadena completa

```
Factura emitida
→ InvoiceIssued
→ GenerateElectronicDocument (listener)
→ ElectronicDocumentGenerator
    └─ InvoiceXmlBuilder      arma el UBL 2.1
    └─ CufeCalculator        el CUFE
    └─ QrContent             el contenido del QR
    └─ XadesSigner           la firma XAdES-EPES
→ electronic_documents (estado FIRMADO)
→ TransmitElectronicDocument (job)
→ DocumentTransmitter
    └─ SoapDianTransport     sobre SOAP + WS-Security
→ document_transmissions
```

Las notas siguen el mismo camino con `NoteDocumentGenerator`, `NoteXmlBuilder` y `CudeCalculator`.

CUFE y CUDE

El **CUFE** es un SHA-384 de 14 campos concatenados. El último es la **clave técnica** de la resolución, más el código de ambiente.

El **CUDE** de las notas es exactamente lo mismo, pero con el **PIN del software** donde el CUFE lleva la clave técnica.

Los dos están verificados contra los ejemplos resueltos del anexo.

Código de seguridad del software

`SoftwareSecurityCode` = `SHA-384(IdSoftware + PIN + NroDocumento)`, según §11.8.

El XML

`InvoiceXmlBuilder` arma UBL 2.1 y lo **valida contra los XSD de la propia DIAN**, versionados en `resources/dian/xsd`. Si no valida, no se guarda.

`erroresDeEsquema($xml)` devuelve la lista de errores; las pruebas la usan.

Excluido y exento no se escriben igual

Clasificación	En el XML
Gravado	<code>cac:TaxTotal</code> con su tarifa
Exento	<code>cac:TaxTotal</code> presente, con <code>TaxAmount</code> y <code>Percent</code> en <code>0.00</code>
Excluido	Sin <code>cac:TaxTotal</code>

Verificado contra los ejemplos oficiales. Confundirlos hace que la DIAN rechace.

La firma

`XadesSigner` reproduce, elemento por elemento, un XML firmado por la propia DIAN.

- Canonicalización **inclusiva** (C14N), no exclusiva.
- Tres referencias: el documento, el `KeyInfo` y las `SignedProperties`.
- Firma RSA-SHA256, resúmenes SHA-384.

El orden importa y no es el obvio. Los resúmenes se calculan sobre los nodos **ya insertados en el documento**: la canonicalización arrastra los espacios de nombres heredados. Calcular `SignedProperties` por separado da un resumen distinto y la firma no cuadra, sin ninguna pista de por qué.

Pendiente: confirmar la URL de la política de firma y su resumen contra la publicada. El anexo menciona una ruta `v2` distinta de la que traen sus propios XML firmados. Si no coinciden, la DIAN rechaza.

El transporte

No hay extensión `soap`: el sobre se arma a mano.

- `SoapDianTransport` — el real. Método `SendBillSync`.
- `FakeDianTransport` — para pruebas.

- `DianTestSetTransport` — el set de pruebas de habilitación.
- `XmlSecuritySigner` — WS-Security, canonicalización **exclusiva** (al revés que la firma del documento), firma Timestamp, `wsa:To` y Body.

Reintentos según el anexo: error → 5 s ×3; timeout → 120 s ×5.

Las URL

`DianEndpoints` las resuelve en tres pasos, del más concreto al más general:

1. `dian_configurations.endpoint_override` de la empresa.
2. `config('dian.endpoint')` — override global del `.env`.
3. La URL pública del ambiente.

Quita el `?wsdl` si alguien lo pegó: esa dirección devuelve la definición del servicio, no el servicio.

El certificado

Se guarda como **archivo** fuera del directorio público; en la base va solo la ruta. Una clave privada en una columna acaba en cada copia de la base.

`SelfSignedCertificate` genera uno autofirmado para pruebas, y `esAutofirmado()` detecta los que la DIAN no va a aceptar comparando emisor contra titular. La marca vive en `dian_certificates.self_signed`.

La representación gráfica

`GraphicRepresentation::para($factura)` devuelve el CUF, el QR como imagen embebida, los datos de la resolución y el emisor. Null si la factura es interna.

Aquí hubo un defecto serio: el CUF estaba **escrito a mano** en la plantilla Blade —el mismo en todas las facturas— y la "fecha de validación" imprimía `created_at`. La causa es la misma en los dos: una plantilla no puede consultar nada. Por eso el bloque se arma en PHP y la vista solo lo pinta.

Diagnóstico y comandos

Comando	Qué hace
<code>dian:diagnostico</code>	Qué falta para poder emitir
<code>dian:certificado-de-pruebas</code>	Genera uno autofirmado (la DIAN no lo acepta)
<code>dian:set-de-pruebas</code>	Manda el set de habilitación
<code>dian:habilitar</code>	Pasa la empresa a producción
<code>dian:transmitir</code>	Reintenta los pendientes
<code>dian:alertas</code>	Avisa de vencimientos
<code>dian:catalogos</code>	Carga los catálogos fiscales

`DianReadiness` es la clase detrás del diagnóstico y del panel. Distingue **bloqueante** de **aviso**: un certificado que caduca en veinte días no impide emitir hoy; uno caducado sí.

Reglas que no se negocian

1. **Un fallo al armar el documento no tumba la emisión.** La factura ya está emitida, el consecutivo ya se gastó y el cliente ya tiene el servicio. El fallo se atrapa, se anota en `last_error` y se sigue.
 2. **Es idempotente.** Volver a generar sobre la misma factura actualiza el documento, no crea otro.
 3. **El ambiente se congela.** Su QR apunta al catálogo de ese ambiente.
-

Red

OLT

- `OltSnmpService` — lecturas por SNMP. Los OID y sus escalas están **verificados contra el equipo real**, no sacados de una MIB.
- `OltSshService` — escrituras. Solo por SSH.
- `OltHardwareDiscovery` — tarjetas, puertos PON y uplinks.
- `OltStatistics` — series de tráfico y potencias.

`onts.rx_power` es **varchar**, no numérico. Ordenar o comparar por esa columna da resultados equivocados. Conviértala.

ONT

`OntPoller` sondea estado y potencia. `OltOntDiscovery` encuentra las que la OLT ve y todavía no están dadas de alta.

PPPoE

`PppoePoller` sondea el router. A diferencia de la ocupación de puertos NAP, aquí el estado **sí se guarda duplicado**: consultar el Mikrotik en cada carga del listado lo haría inusable.

`PppoeMassCutoff` hace los cortes masivos, en dos pasos y sin tocar el estado del contrato.

Planta de fibra

`FiberPlantManager` y `FiberPathTracer`. Un hilo tiene dos extremos, y el impacto de un corte se calcula **por simulación** recorriendo el camino, no leyendo una columna.

La ocupación de un puerto NAP **se deduce**, nunca se guarda: un puerto marcado como libre que tiene un cliente es peor que consultarlo.

Trazabilidad

Cómo se registra

El trait `App\Billing\Concerns\Auditable` en un modelo hace que cada `created/updated/deleted` escriba en `audits`, con los valores antes y después de **solo lo que cambió**.

```
use App\Billing\Concerns\Auditable;

class MiModelo extends Model
{
    use Auditable;
}
```

Delega en `AuditLogger`, que añade el contexto común: usuario, IP, sucursal, rol, ruta y sesión.

Datos sensibles

`config/audit.php` → `redacted_attributes` lista lo que se tapa: contraseñas, tokens, comunidades SNMP, `software_pin`, `technical_key`.

Si añade una columna con un secreto, añádala a esa lista.

Sesiones

`user_sessions` es una tabla propia. La correlación se hace por `_trace_session_id` guardado en la sesión, no por el `session_id` de Laravel.

Lo que NO se audita

La telemetría de los sondeos de red. Son decenas de miles de filas por día que tapan lo que importa.

Deuda conocida: la tabla `audits` está en el orden de 2,67 GB sin política de retención. Existe `audits:prune` pero no está en el planificador.

Notificaciones

Canales: correo y WhatsApp. El de WhatsApp es intercambiable (arranca simulado; la implementación Meta se activa con configuración).

Los correos usan **una sola plantilla HTML** para los ocho mensajes. Reglas de correo: maquetar con tablas, estilos en línea, nada de CSS externo.

Hay siete disparadores en cola. El badge de órdenes de los técnicos funciona por sondeo.

Pruebas

Cómo correrlas

```
php artisan test
php artisan test tests/Feature/Billing/InvoiceGenerationTest.php
php artisan test --filter=Dian
```

Son **104 archivos y unas 1.290 pruebas**. La suite completa tarda alrededor de 30 minutos.

Reglas

- **Nunca edite código mientras la suite corre.** Produce fallos falsos que cuestan mucho tiempo diagnosticar. Deténgala primero.
- Las pruebas están en español y su nombre describe la regla, no el método.
- El bloque de comentario de cada clase de prueba explica **qué defiende**, no qué prueba.
- Use `RefreshDatabase`. `BillingTestCase` prepara roles, permisos y la cadena sucursal → cliente → plan → contrato.

Qué se prueba de verdad

No el CRUD. Lo que se defiende es lo que duele si se rompe: que una sucursal no vea datos de otra, que un documento interno no gaste un consecutivo autorizado, que el certificado no se pueda descargar, que un fallo del XML no tumbe la emisión.

Tareas programadas y cola

El planificador

El servidor necesita **una sola línea de cron** llamando a `schedule:run`. Todo lo demás está en `app/Console/Kernel.php`.

Tarea	Cuándo
<code>onts:poll, olt:poll-ports, pppoe:poll</code>	cada 5 minutos
<code>olt:discover-ports</code>	03:15
Recortes de telemetría	03:30 - 04:00
<code>sessions:sweep</code>	cada hora
<code>invoices:notify-reminders</code>	08:00
<code>dian:alertas</code>	07:00
<code>dian:transmitir</code>	cada hora

Las copias de seguridad **no** están aquí: las lanza el cron del sistema, a propósito.

La cola

`QUEUE_CONNECTION=database` en producción. Hace falta un worker:

```
php artisan queue:work --tries=3
```

Sin worker, los documentos electrónicos se firman pero **no se transmiten** hasta que `dian:transmitir` los recoja en la siguiente hora.

Copias de seguridad

El diseño es deliberado: **PHP vuelca, bash transporta**.

`RunBackup` genera el volcado; el traslado y la rotación los hace un script de sistema. No hay tabla de copias: el listado se arma leyendo el directorio, validado por `BackupRepository::buscar()`.

Está fuera del planificador de Laravel a propósito, para que una copia no dependa de que la aplicación esté sana.

Trampas conocidas

Errores que ya se cometieron aquí. Están documentados para no repetirlos.

`validate()` no devuelve las claves ausentes. `$datos['campo']` revienta con "Undefined array key" si el campo es opcional y no vino. Use `(isset($datos['x']) ?? null) ?: null`.

Laravel inyecta un modelo VACÍO, no null. Cuando una ruta no trae el parámetro, el modelo llega instanciado y sin existir — y un objeto vacío es **truthy**. Use `$m?->exists ? $m : null`.

`withBody()` pisa el `Content-Type` de `withHeaders()`. En las peticiones a la DIAN eso borraba el parámetro `action` del content type. Pase el tipo completo a `withBody()`.

Código muerto después de `return DB::transaction(...)`. Todo lo que venga después no se ejecuta. Capture el resultado: `$x = DB::transaction(...)`.

Un campo que falta en `\$fillable` se descarta en silencio. La asignación masiva no avisa. Si un dato "no se guarda" y no hay error, mire ahí primero.

`hasMany` donde iba `belongsTo`. Revienta con "Unknown column" solo cuando algo hace **eager load** de la relación. Puede pasar mucho tiempo escondido.

Un `@php` en Blade con comillas escapadas compila y revienta al renderizar.

Los datos de `fake()` pueden llevar apóstrofes. Al comparar HTML escapado, use valores explícitos en vez de `fake()`.

Mapa: dónde tocar qué

Quiero...	Vaya a
Cambiar cómo se arma una factura	<code>app/Billing/Services/InvoiceGenerator.php</code>
Cambiar la numeración	<code>app/Billing/Services/InvoiceNumerator.php</code> , <code>app/Services/Numbering/DocumentNumberService.php</code>
Tocar el XML de la DIAN	<code>app/Billing/Dian/InvoiceXmlBuilder.php</code>
Tocar la firma	<code>app/Billing/Dian/XadesSigner.php</code>
Tocar el envío a la DIAN	<code>app/Billing/Dian/Transport/</code>
Cambiar el PDF de la factura	<code>resources/views/gestisp/invoices/pdf.blade.php</code> y <code>partials/dian.blade.php</code>
Añadir un permiso	<code>database/seeder/RoleSeeder.php</code> , luego <code>php artisan permissions:sync</code>
Cambiar el menú	La configuración de AdminLTE y <code>RoleBasedMenuFilter</code>
Añadir un informe	<code>app/Reports/</code>
Tocar el filtrado multiempresa	<code>app/Tenancy/</code>

El menú

Lo filtra **solo** `RoleBasedMenuFilter`, por el rol de la sesión. Hubo también un filtro por `*gates*` que se retiró porque duplicaba la decisión. No lo reintroduzca.

Deuda técnica y pendientes

La lista viva está en `docs/Pendientes.md`. Lo principal:

Bloqueantes de facturación electrónica

1. **El certificado en archivo `.p12`**. El único que no depende de programar.
2. **Confirmar la política de firma** contra la publicada.
3. **Probar la firma contra un certificado real**.

A medias

- **Contingencia tipo 04**. La situación se detecta (`DocumentTransmitter::agotoLosIntentos()`) pero no se actúa: no se reemite con el código 04 ni se lleva el plazo de 48 horas.
- **Consulta por `trackId`**. No se consulta el estado de un documento ya enviado.
- **AttachedDocument**. El XSD está en `resources/dian/xsd`, el constructor no existe. Es el envoltorio que se le entrega al adquirente con el XML y la respuesta de la DIAN.
- **Entrega del XML al cliente**. El correo de factura no adjunta ni el XML ni el PDF.

Otras

- **`audits` sin retención**, en el orden de 2,67 GB.
 - **Planes y servicios no se comparten entre sucursales**. Ambos están atados a una `branch_id` y se duplican por sede. Es la decisión D8 del plan maestro, escrita y nunca implementada. Migrar exige decidir qué hacer con los ya duplicados.
 - **Informes en modo consolidado**: algunas cifras son aproximaciones sobre columnas que no fueron diseñadas para eso.
-

Documentación relacionada

En [docs/](#):

Archivo	Tema
Plan-Empresa-Sucursales-Grupos.md	El plan maestro y las decisiones D1-D9
Pendientes.md	Lo que falta, con el porqué
Documento-Electronico.md	XML, CUFE, QR, firma
Transmision-DIAN.md	El transporte SOAP
Notas-Electronicas.md	Notas crédito y débito
Certificado-Digital.md	Qué certificado pedir y por qué
Habilitacion-DIAN.md	De cero a producción
Clasificacion-Fiscal.md	Gravado, excluido y exento
Numeracion.md	Las series y los rangos
Multiempresa.md	Empresa, sucursal y contexto
Datos-Fiscales.md	Catálogos y completitud